

Typed λ -calculus

1 Untyped λ -calculus and its problems

Recap: we have been looking at λ -calculus with arithmetic. The syntax is as follows.

$$\begin{aligned} M ::= & \quad x \ (x \in \text{Vars}) \\ & \quad | \ \lambda x.M \ (x \in \text{Vars}) \\ & \quad | \ MM \\ & \quad | \ \underline{n} \ (n \in \mathbb{Z}) \\ & \quad | \ M + M \ | \ M - M \ | \ M \times M \\ & \quad | \ (M) \end{aligned}$$

To evaluate a term, we look inside it for a *redex* e.g. $\underline{m} + \underline{n}$ or $(\lambda x.M)N$.

This is *untyped* λ -calculus, i.e. there are no types. The intended meaning of $\lambda x.x$ is “the function sending anything to itself”. We can form terms such as the following:

$$\begin{aligned} & (\lambda x.M) + \underline{n} \\ & \underline{n} + (\lambda x.M) \\ & (\lambda x.M) + (\lambda y.N) \\ & \underline{n}M \end{aligned}$$

Let’s call these “badexes”. They make no sense, but they are correct terms of our λ -calculus, as is $\Omega \stackrel{\text{def}}{=} (\lambda x.xx)(\lambda y.yy)$.

Recall that, because of the Church-Rosser theorem, a term has at most one $\beta\delta$ -normal form. It’s not possible to get two different answers (normal forms) from one term. But consider the following term:

$$(\lambda x.\underline{3})(\underline{17}\lambda y.y)$$

It has normal form $\underline{3}$ but informally this illustrates that “whether something goes wrong may depend on the evaluation order”. A programming language that evaluates the argument to a function first (“call-by-value”, like OCaml or Java) would get stuck when evaluating this term. Likewise consider the following term:

$$(\lambda x.\underline{3})\Omega$$

A call-by-value programming language evaluating this term will run forever.

2 Introducing Types

Let’s consider the following syntax of types

$$A ::= \text{int} \ | \ A \rightarrow A \ | \ (A)$$

For example $\text{int} \rightarrow \text{int}$ is the type of functions that take an integer argument and return an integer. The convention is that $\rightarrow$ associates to the right. For example $\text{int} \rightarrow \text{int} \rightarrow \text{int}$ is $\text{int} \rightarrow (\text{int} \rightarrow \text{int})$.

The syntax of terms is slightly different from before:

$$\begin{aligned} M ::= & \quad x \ (x \in \text{Vars}) \\ & \quad | \ \lambda x_A.M \ (x \in \text{Vars}) \\ & \quad | \ MM \\ & \quad | \ \underline{n} \ (n \in \mathbb{Z}) \\ & \quad | \ M + M \ | \ M - M \ | \ M \times M \\ & \quad | \ (M) \end{aligned}$$

Note that λx is always annotated with a type.

We can give formal typing rules saying when a term has a given type. You can learn about these in Principles of Programming Languages.

Many programming languages, e.g. OCaml, Haskell, Java are typed. Others, like Scheme and Lisp, are not.

3 Type Inference

Example Here is an untyped term:

$$\lambda x.x\texttt{3} + \texttt{7}$$

Annotate it with types.

To answer this, we have to look for the clues.

1. x is applied to something. So it has type $A \rightarrow B$.
2. The thing it's applied to must have type A , but it's an integer. So $A = \texttt{int}$.
3. $x\texttt{3}$ has type B , and gets added to something. So $B = \texttt{int}$.

In conclusion, x has type $\texttt{int} \rightarrow \texttt{int}$. So the annotated term is

$$\lambda x_{\texttt{int} \rightarrow \texttt{int}}.x\texttt{3} + \texttt{7}$$

which has type $(\texttt{int} \rightarrow \texttt{int}) \rightarrow \texttt{int}$.

Example Here is an untyped term:

$$\lambda x.\lambda y.x(xy) + \texttt{17}$$

Annotate it with types.

To answer this, we have to look for the clues.

1. x is applied to something, so it has type $A \rightarrow B$.
2. x is applied to y , so y has type A .
3. x is applied to xy , which has type B . So $A = B$.
4. $x(xy)$, which has type B , is added to something, viz. 17. So $B = \texttt{int}$.

In conclusion, x has type $\texttt{int} \rightarrow \texttt{int}$ and y has type $\texttt{int}$. So the annotated term is

$$\lambda x_{\texttt{int} \rightarrow \texttt{int}}.\lambda y_{\texttt{int}}.x(xy) + \texttt{17}$$

This has type

$$(\texttt{int} \rightarrow \texttt{int}) \rightarrow (\texttt{int} \rightarrow \texttt{int})$$

or just

$$(\texttt{int} \rightarrow \texttt{int}) \rightarrow \texttt{int} \rightarrow \texttt{int}$$

Example Here is an untyped λ -term

$$(\lambda x.\texttt{3})(\texttt{17}\lambda y.y)$$

Annotate with types if possible.

We see that $\texttt{17}$ is applied to something, so it has type $A \rightarrow B$, but it also has type $\texttt{int}$. Contradiction.

Example Here is an untyped λ -term:

$$(\lambda x.xx)(\lambda y.yy)$$

Annotate with types if possible.

Let's look for the clues.

1. x gets applied to something, so it has type $A \rightarrow B$.
2. x gets applied to x , so $A = A \rightarrow B$.

So A is $((\dots \rightarrow B) \rightarrow B) \rightarrow B$. But our type expressions are finite. Contradiction.

Example Here is an untyped λ -term:

$$\lambda x. x \underline{7}$$

Annotate with types as generally as possible.

Let's look for the clues.

1. x is applied to something, so it has type $A \rightarrow B$.
2. x is applied to $\underline{7}$, which has type int , so $A = \text{int}$.

We conclude that x has type $\text{int} \rightarrow B$. No more information is available. So the most general annotated term is

$$\lambda x_{\text{int} \rightarrow B}. x \underline{7}$$

which has type $(\text{int} \rightarrow B) \rightarrow B$.

4 Multiary functions

In λ -calculus (at least, the version we've seen), all functions are *unary*, i.e. they take just one argument. How can we express multiary functions (functions with several arguments) in λ -calculus?

We use a trick called *currying* (named after Haskell Curry). We represent a function with two arguments as a function that returns a function. For example, suppose we want to write the function that takes arguments x_{int} and y_{int} to $x + 17 \times y$. We represent it as

$$\lambda x_{\text{int}}. \lambda y_{\text{int}}. x + 17 \times y$$

which has type $\text{int} \rightarrow (\text{int} \rightarrow \text{int})$. Here's a term that uses it:

$$(\lambda f_{\text{int} \rightarrow (\text{int} \rightarrow \text{int})}. (f 5) 2) (\lambda x_{\text{int}}. \lambda y_{\text{int}}. x + 17 \times y)$$

This is the reason for the conventions we've learnt:

- $\rightarrow$ associates to the right
- application associates to the right.

We also use currying in untyped λ -calculus.

$$\begin{aligned} (\lambda f. f \ 5 \ 2) (\lambda x. \lambda y. x + 17 * y) &\rightsquigarrow \\ (\lambda x. \lambda y. x + 17 * y) \ 5 \ 2 &\rightsquigarrow \\ (\lambda y. 5 + 17 * y) \ 2 &\rightsquigarrow \\ 5 + 17 * 2 &\rightsquigarrow \\ 5 + 34 &\rightsquigarrow \\ 39 & \end{aligned}$$

5 Advantages and disadvantages of types

As stated, some programming languages are typed and some are not.

- + In typed λ -calculus, there are no terms that evaluate to the point where they contain a “badex” such as $(\lambda x. 3) + 7$. That is because β -reduction and δ -reduction preserve typeability, and badexes don't typecheck. To summarize: *well-typed programs don't go wrong*. This enables many bugs to be caught at compile-time, so they don't cause damage.
- + In typed λ -calculus, there are no terms that run forever. To be more precise, the *strong normalization* theorem says that there is no infinite sequence of reductions

$$M_0 \rightsquigarrow M_1 \rightsquigarrow M_2 \rightsquigarrow \dots$$

for typed λ -calculus with arithmetic. However, we often want to extend typed λ -calculus with features that allow terms to run forever, e.g. recursion or iteration (while loop).

- Types rule out useful programs. Consider for example, the following term in untyped λ -calculus:

$$(\lambda f.(f \lambda x.x)(f \underline{3})) \lambda y.y$$

Its reduces in finitely many steps with no badexes:

$$\begin{array}{c}
 (\lambda f.(f \lambda x.x)(f \underline{3})) \lambda y.y \\
 \downarrow \\
 ((\lambda y.y)(\lambda x.x))((\lambda y.y)\underline{3}) \longrightarrow (\lambda x.x)((\lambda y.y)\underline{3}) \\
 \downarrow \qquad \qquad \qquad \downarrow \\
 ((\lambda y.y)(\lambda x.x))\underline{3} \longrightarrow (\lambda x.x)\underline{3} \\
 \qquad \qquad \qquad \downarrow \\
 \qquad \qquad \qquad \underline{3}
 \end{array}$$

Can we typecheck it? Let's look for the clues.

1. f is applied to something, so it has type $A \rightarrow B$.
2. f is applied to $\underline{3}$ so $A = \text{int}$.
3. f is applied to $\lambda x.x$ so $A = C \rightarrow D$.

This is a contradiction, so the term doesn't typecheck. To allow terms like this, we need a fancier type system. (Specifically, a polymorphic one.) And no matter how fancy the type system, there will always be legitimate terms that are ruled out. This is because whether a term is safe (avoids badexes and has no infinite reduction sequences) is undecidable.

Some languages have a compromise solution called *soft typing*: types are present most of the time, but it is possible to override the typing constraints.

- + Typed terms have a meaning. For example, the meaning of $\lambda x_{\text{int}}.x$ is the identity function $\mathbb{Z} \rightarrow \mathbb{Z}$. The meaning of $\lambda x_{\text{bool}}.x$ is the identity function $\mathbb{B} \rightarrow \mathbb{B}$ is the identity function. But the intended meaning of $\lambda x.x$ is “the function sending *anything* to itself”, which doesn't exist according to the standard rules of set theory, developed in the early 20th century to avoid problems such as Russell's paradox. So, to give meaning to untyped λ -calculus, more advanced techniques are required. The first solution to this problem was given by Dana Scott in the 1960s.

6 Exercises

Here is a type grammar:

$$\tau ::= \text{int} \mid \text{bool} \mid \tau \rightarrow \tau \mid (\tau)$$

For each of the following expressions, say whether it is typeable. If so, say what is the most general type, and annotate the term with types.

1. $\lambda x. \lambda y. (x + y) > 2$
2. $\lambda x. \lambda y. x (y \ 3)$
3. $\lambda x. x \ 3 + x$
4. $\lambda x. x (3 + x)$
5. $\lambda x. \lambda y. \lambda z. x (y \ 3) + x(z \ \text{true})$
6. $\lambda x. \lambda y. x \ y$